Week 4 Internship Report: API Integration and Data Handling

Project: WeatherWise — Modern Real-Time Weather Dashboard

Assignment	Week 4: API Integration and Data Handling
Track	Frontend Architecture & React.js Web Development
Student Name	[Student Name]
Institution	[Institution / Organization Name]
Submission Date	October 9, 2026
Tech Stack	React 18, Vite 6, Vanilla CSS3, Lucide React, Open-Meteo APIs

Project: WeatherWise — Modern Real-Time Weather Dashboard

Cover Page Details

• **Project Title:** WeatherWise — Modern Real-Time Weather Dashboard

• **Assignment Name:** Week 4: API Integration and Data Handling in React.js

• **Track:** Frontend Architecture & React.js Web Development

• **Student Name:** *[Student Name]*

• **Institution / Organization:** *[Institution / Organization Name]*

• **Submission Date:** October 9, 2026

• **Technology Stack:** React.js 18, Vite 6, JavaScript (ES6+), Vanilla CSS3, Lucide React, Open-Meteo Geocoding & Forecast APIs

---

1. Introduction

Modern web applications rely heavily on external Application Programming Interfaces (APIs) to deliver real-time, dynamic information to end users. In frontend development with React.js, mastering API integration involves far more than simply issuing a network request; it demands resilient state management, asynchronous control flow, request lifecycle handling (including cancellation), input sanitization, data transformation, and responsive visual feedback.

This report documents the design, architecture, and implementation of **WeatherWise**, a weather dashboard application developed for the Week 4 ReactJS internship assignment titled **“API Integration and Data Handling.”**

---

2. Project Objectives

The core objectives of the WeatherWise project are:

1. **Practical API Integration:** Integrate external public RESTful APIs without requiring secret API keys or paid authentication tiers.

2. **Asynchronous JavaScript Mastery:** Utilize native `async/await` and the `Fetch API` for predictable asynchronous data pipelines.

3. **React State & Lifecycle Management:** Structure state with `useState` and manage side effects with `useEffect` and custom hooks (`useWeather`).

4. **Resilient Data Handling:** Implement robust input validation, response status verification (`response.ok`), error boundaries, and request cancellation via `AbortController`.

5. **Data Transformation & Normalization:** Map raw World Meteorological Organization (WMO) weather interpretation codes to intuitive human labels and dynamic icons.

6. **Dynamic Unit Conversion:** Provide instant bidirectional conversion between metric (Celsius, km/h, mm) and imperial (Fahrenheit, mph, in) units without re-fetching data.

7. **Production-Ready UI/UX:** Deliver a glassmorphic user interface with responsive layouts for mobile, tablet, and desktop screens.

---

3. Problem Statement

Weather applications often suffer from multiple frontend architectural flaws:

• **Overfetching & Stale Data:** Uncoordinated network requests can lead to race conditions where older requests overwrite newer search results.

• **Poor Error Feedback:** Users are frequently left with blank screens or cryptic technical error messages when network drops or invalid city queries occur.

• **Cluttered State Architecture:** Mixing data fetching logic directly inside visual presentation components leads to tight coupling and unmaintainable code.

• **Unnecessary Re-renders & Request Duplication:** Toggling display units (such as Celsius to Fahrenheit) often causes redundant API calls when done naively.

WeatherWise resolves these challenges by decoupling the API service layer, custom hook state orchestration, and pure presentation components.

---

4. Overview of WeatherWise

WeatherWise is a full-featured, responsive weather dashboard that allows users to search for any city globally, view instant atmospheric conditions, explore 24-hour hourly forecasts, and inspect 7-day extended projections.

Key Capabilities:

• **Real-Time City Geocoding & Disambiguation:** Locates matching cities globally and displays disambiguation options when multiple locations share the same name (e.g., Paris, France vs. Paris, Texas, USA).

• **Atmospheric Condition Metrics:** Displays temperature, feels-like temperature, humidity percentage with visual meter bars, wind velocity, precipitation depth, UV Index risk categorization, and sunrise/sunset schedules.

• **24-Hour Hourly Timeline:** A smooth, horizontally scrollable strip showing upcoming temperatures and precipitation probabilities starting from the current hour.

• **7-Day Daily Forecast:** Extended forecast with relative temperature range indicator bars and daily UV exposure ratings.

• **Unit Switching:** Zero-latency toggling between Celsius (°C) and Fahrenheit (°F).

• **Recent Searches:** Session-cached history chips allowing one-click retrieval of previously searched cities.

• **Live Refresh:** Instant re-synchronization with Open-Meteo meteorological models.

---

5. Technologies and Libraries Used

```
| Technology | Version | Purpose |
| :--- | :--- | :--- |
| **React.js** | `^18.3.1` | Component-based UI library, Hooks (`useState`, `useEffect`, `useCallback`, `useRef`) |
| **Vite** | `^6.0.3` | Next-generation frontend build tool providing lightning-fast HMR and optimized bundling |
| **JavaScript (ES6+)** | Modern | Asynchronous programming (`async/await`, Promises), destructuring, modules |
| **Vanilla CSS3** | Custom | Responsive design system, CSS variables, Glassmorphism, animations |
| **Lucide React** | `^0.468.0` | Accessible, tree-shakeable SVG weather and UI iconography |
| **Open-Meteo APIs** | Public v1 | Free, high-resolution geocoding and meteorological forecast datasets |
```

---

6. Why Open-Meteo Was Selected

Open-Meteo was chosen for this project for several critical technical advantages:

1. **Open & Non-Commercial Friendly:** Does not require proprietary API keys, credit cards, or rate-limited auth tokens, making it ideal for evaluation and open-source demonstration.

2. **High Precision & Global Coverage:** Combines global numerical weather prediction models (ECMWF, DWD ICON, NOAA GFS) with high-resolution hourly and daily variables.

3. **Structured JSON Output:** Returns clean, standard numeric arrays and WMO weather codes.

4. **Integrated Geocoding:** Provides a dedicated search API that resolves city names to exact latitude, longitude, timezone, and country data.

---

7. API Endpoints and Their Purposes

A. Geocoding API (City Search)

• **Endpoint:** `https://geocoding-api.open-meteo.com/v1/search`

• **Query Parameters:**

• `name`: Search query string (e.g., `"Tokyo"`, `"London"`)

• `count`: Maximum results to return (`5`)

• `language`: Result language (`en`)

• `format`: Response format (`json`)

• **Purpose:** Resolves text queries into geographic coordinates, administrative regions, country names, and local timezones.

B. Forecast API (Weather Conditions & Projections)

• **Endpoint:** `https://api.open-meteo.com/v1/forecast`

• **Query Parameters:**

• `latitude`: Location latitude (e.g., `51.5085`)

• `longitude`: Location longitude (e.g., `-0.1257`)

• `current`: `temperature_2m,relative_humidity_2m,apparent_temperature,is_day,precipitation,weather_code,wind_speed_10m`

• `hourly`: `temperature_2m,precipitation_probability,weather_code`

• `daily`: `weather_code,temperature_2m_max,temperature_2m_min,sunrise,sunset,uv_index_max`

• `timezone`: Location timezone or `auto`

• `forecast_days`: `7`

• **Purpose:** Provides current atmospheric metrics, 24-hour hourly trendlines, and 7-day daily projections in a single optimized payload.

---

8. Application Architecture and Folder Structure

The project follows a clean separation of concerns:

```
weatherwise-week4-api-integration/
■■■ index.html
■■■ package.json
■■■ vite.config.js
■■■ README.md
■■■ screenshots/
■■■ src/
■■■ main.jsx # Entry point mounting React DOM
■■■ App.jsx # Top-level view coordinator
■■■ index.css # Design system & responsive rules
■■■ components/
■ ■■■ Header.jsx # App branding, live clock, unit toggle
■ ■■■ SearchBar.jsx # Input field, clear button, city dropdown
■ ■■■ RecentSearches.jsx # Recent search chips
■ ■■■ CurrentWeather.jsx # Hero weather overview card
■ ■■■ WeatherDetails.jsx # Atmospheric metrics grid
■ ■■■ HourlyForecast.jsx # 24-hour horizontal scroll forecast
■ ■■■ DailyForecast.jsx # 7-day extended forecast
■ ■■■ UnitToggle.jsx # Celsius/Fahrenheit toggle buttons
■ ■■■ WeatherIcon.jsx # Dynamic Lucide icon mapper
■ ■■■ LoadingState.jsx # Shimmer skeleton loader
■ ■■■ ErrorMessage.jsx # Error banner with retry/dismiss
■ ■■■ EmptyState.jsx # Welcome view with suggested cities
■■■ hooks/
■ ■■■ useWeather.js # Custom hook orchestrating all weather state
■■■ services/
■ ■■■ weatherApi.js # Low-level Fetch layer & response transformer
■■■ utils/
■■■ weatherCodes.js # WMO weather code mapping & descriptions
■■■ temperature.js # Unit conversions & UV categorization
■■■ dateFormat.js # Timezone-aware date & time formatters
```

---

9. Explanation of How the City Search API Works

1. **User Input & Validation:** When the user enters a city name in `SearchBar.jsx`, the query is trimmed. If empty or fewer than 2 characters, the user is immediately alerted without sending a network request.

2. **Asynchronous Request:** `searchCities(query)` in `weatherApi.js` builds a safely URL-encoded request to the Open-Meteo Geocoding endpoint.

3. **Response Validation:** The service checks `response.ok` and verifies that the `results` array exists.

4. **Disambiguation Handling:**

• If **0 results** are found, an intuitive "No locations found" message is displayed.

• If **1 result** is found, the application immediately fetches the weather for that location.

• If **multiple results** are found, the top result is pre-loaded while a disambiguation dropdown displays the candidate locations with administrative regions and country codes for the user to select.

---

10. Explanation of How Weather Data Is Fetched

1. **Coordinate-Based Querying:** Once coordinates (`latitude`, `longitude`, `timezone`) are identified, `fetchWeatherForecast()` queries the Open-Meteo Forecast endpoint.

2. **Single Comprehensive Fetch:** Current weather, 24-hour hourly forecast, and 7-day daily forecast are fetched in one network roundtrip.

3. **Data Transformation:** The raw Open-Meteo response is passed through `transformWeatherData()` to create a clean, normalized JavaScript object:

• Current weather values are mapped directly.

• The hourly forecast is filtered to start from the current hour and extend for the next 24 hours.

• Daily forecast records are formatted with high/low temperatures, UV index, and sunrise/sunset timestamps.

---

11. Asynchronous Operations Using `async/await`

All asynchronous operations in WeatherWise use modern `async/await` syntax for readability and predictability:

```
export async function fetchWeatherForecast(latitude, longitude, timezone = 'auto', signal) {
const queryParams = new URLSearchParams({
latitude: latitude.toString(),
longitude: longitude.toString(),
current: 'temperature_2m,relative_humidity_2m,apparent_temperature,is_day,precipitation,weather_co
de,wind_speed_10m',
hourly: 'temperature_2m,precipitation_probability,weather_code',
daily: 'weather_code,temperature_2m_max,temperature_2m_min,sunrise,sunset,uv_index_max',
timezone: timezone || 'auto',
forecast_days: '7'
});

const url = `${FORECAST_BASE_URL}?${queryParams.toString()}`;

try {
const response = await fetch(url, { signal });

if (!response.ok) {
throw new Error(`Weather service error: ${response.status} ${response.statusText}`);
}
```

```
const rawData = await response.json();
return transformWeatherData(rawData);
} catch (error) {
if (error.name === 'AbortError') {
throw error; // Propagate aborts cleanly
}
throw new Error(error.message || 'Failed to retrieve weather data.');
}
}
```

---

12. React State Management & The `useWeather` Custom Hook

Rather than cluttering components with state variables, WeatherWise encapsulates all business logic within a custom hook named `useWeather`:

• **`selectedLocation`**: Stores the active city object (`{ name, country, admin1, latitude, longitude, timezone }`).

• **`weatherData`**: Holds normalized weather data for current, hourly, and daily displays.

• **`loading` & `searching`**: Boolean flags controlling skeleton loaders and search spinners.

• **`error`**: Holds user-friendly error strings or `null`.

• **`unit`**: Active temperature unit (`'C'` or `'F'`).

• **`recentSearches`**: Array of the last 5 unique searched locations, synced to `sessionStorage`.

• **`abortControllerRef`**: `useRef` keeping track of the current in-flight request to allow instantaneous cancellation.

---

13. How API Responses Are Processed and Rendered

1. **WMO Weather Code Interpretation:** Open-Meteo returns integer weather codes (e.g., `0` for Clear Sky, `61` for Slight Rain, `95` for Thunderstorm). The `weatherCodes.js` utility maps each code to a human-readable title, a descriptive subtitle, and dynamic day/night Lucide icons (`Sun`, `Moon`, `CloudRain`, `CloudSnow`, `CloudLightning`).

2. **Date and Time Formatting:** `dateFormat.js` uses standard `Intl.DateTimeFormat` to format dates into friendly labels ("Today", "Tomorrow", "Wed, Oct 14") and hours into 12-hour AM/PM formats.

3. **UV Index Categorization:** `temperature.js` classifies UV index values into risk categories: Low (<3), Moderate (3–5), High (6–7), Very High (8–10), and Extreme (11+) with tailored badge colors.

---

14. Loading States and Error-Handling Strategies

• **Shimmer Skeleton Loading:** When weather data is loading, `LoadingState.jsx` renders animated skeleton cards matching the dimensions of the hero card, metrics grid, and forecast rows to prevent layout shifts.

• **Descriptive Error Banners:** `ErrorMessage.jsx` provides an error card with a retry button (`refreshWeather`) and a dismiss button (`clearError`).

• **Defensive Property Access:** All components check for nullish data before rendering properties (e.g., `todayDaily?.uvIndexMax ?? '--'`).

---

15. Request Cancellation & Prevention of Stale Responses

When a user searches multiple cities rapidly, network responses may return out of order. WeatherWise resolves this using `AbortController`:

```
// Cancel any ongoing fetch before initiating a new one
if (abortControllerRef.current) {
abortControllerRef.current.abort();
}
abortControllerRef.current = new AbortController();

const data = await fetchWeatherForecast(
location.latitude,
location.longitude,
location.timezone,
abortControllerRef.current.signal
);
```

When an abort occurs, the resulting `AbortError` is caught and ignored, preventing older network requests from overwriting newer user selections.

---

16. Temperature and Unit Conversion Logic

Temperature conversion is performed dynamically in memory without triggering duplicate network requests:

• **Celsius to Fahrenheit Formula:** `F = (C * 9/5) + 32`

• **Wind Speed Conversion:** `1 km/h = 0.621371 mph`

• **Precipitation Conversion:** `1 mm = 0.0393701 in`

When the user switches the unit toggle, React re-renders the formatted numbers instantly while preserving all existing weather data.

---

17. Responsive Design & Performance Optimizations

1. **Responsive CSS Grid & Flexbox:**

• **Desktop (> 1024px):** Two-column top layout (Hero Weather on the left, 2x2 Atmospheric Metrics on the right) with full horizontal forecast scroll.

• **Tablet (640px – 1024px):** Single-column stacked layout with adapted grid columns.

• **Mobile (< 640px):** Single-column stacked cards, compact search bar, full-width buttons, and 2-row daily forecast cards.

2. **Performance Optimizations:**

• **Zero Over-Fetching:** Single query fetches current, hourly, and daily data simultaneously.

• **No State Duplication:** Metric conversions happen on-the-fly during rendering.

• **Debounced/Clean Effects:** Stable callback dependencies via `useCallback` prevent unwanted re-renders.

---

18. Important Code Samples

A. Geocoding Search & Disambiguation (`src/services/weatherApi.js`)

```
export async function searchCities(cityName, signal) {
const trimmed = cityName ? cityName.trim() : '';
if (!trimmed) return [];

const url =
`${GEOCODING_BASE_URL}?name=${encodeURIComponent(trimmed)}&count=5&language=en&format=json`;

const response = await fetch(url, { signal });
if (!response.ok) {
throw new Error(`Geocoding server error: ${response.status}`);
}

const data = await response.json();
if (!data.results || data.results.length === 0) return [];

return data.results.map((item) => ({
id: `${item.id || item.latitude}_${item.longitude}`,
name: item.name,
latitude: item.latitude,
longitude: item.longitude,
country: item.country || '',
countryCode: item.country_code || '',
admin1: item.admin1 || '',
timezone: item.timezone || 'auto'
}));
}
```

B. Weather Code Interpretation (`src/utils/weatherCodes.js`)

```
export function getWeatherInterpretation(code, isDay = 1) {
const isDaytime = isDay === 1 || isDay === true;
const match = WEATHER_CODE_MAP[code];

if (!match) {
return {
code,
label: 'Partly Cloudy',
iconName: isDaytime ? 'CloudSun' : 'CloudMoon',
description: 'Typical mixed sky conditions.'
};
}

return {
code,
label: match.label,
iconName: isDaytime ? match.dayIcon : match.nightIcon,
description: match.description
};
}
```

---

19. Challenges Encountered and Solutions

1. **Challenge: Timezone Offset Discrepancies in Daily Dates**

• *Problem:* Converting raw ISO date strings like `"2026-10-09"` with `new Date()` could result in previous-day dates in western timezones due to UTC offset shifts.

• *Solution:* Parsed date strings explicitly (`isoDate.split('-')`) before constructing local date representations in `dateFormat.js`.

2. **Challenge: Disambiguating Multiple Cities with Identical Names**

• *Problem:* A search for "Paris" could refer to Paris (France) or Paris (Texas, USA).

• *Solution:* Added a disambiguation dropdown in `SearchBar.jsx` showing the administrative region and country code so the user can easily select their intended location.

3. **Challenge: Preventing Rapid Search Overwrites (Race Conditions)**

• *Problem:* Fast consecutive keystrokes or searches could result in slower earlier requests finishing after faster later requests.

• *Solution:* Implemented `AbortController` in `useWeather.js` to cancel pending requests upon new searches.

---

20. Testing Procedures and Verification Results

```
| Test Case | Description | Expected Outcome | Actual Result | Status |
| :--- | :--- | :--- | :--- | :--- |
| **TC-01** | Initial Page Load | Loads default city (London) with live weather, metrics, and forecasts | Loaded London with live conditions | **PASS** |
| **TC-02** | Valid City Search ("Tokyo") | Fetches Tokyo coordinates, updates hero card, 24h forecast, and 7-day forecast | Successfully updated to Tokyo | **PASS** |
| **TC-03** | City Disambiguation ("Paris") | Displays disambiguation dropdown with matching regions | Displayed Paris (France) and other matches | **PASS** |
| **TC-04** | Invalid City Search ("xyznonexistent") | Shows user-friendly error notice stating no locations found | Displayed error notice; UI remained intact | **PASS** |
| **TC-05** | Empty Input Submission | Prevents request submission and prompts user for valid input | Blocked submission with error feedback | **PASS** |
| **TC-06** | Unit Toggle (°C ↔ °F) | Updates all temperatures, wind speeds, and precipitation without re-fetching | Converted units seamlessly in real time | **PASS** |
| **TC-07** | Live Refresh Button | Triggers fresh API fetch with spinning refresh icon animation | Refreshed data and updated timestamp | **PASS** |
| **TC-08** | Recent Searches History | Adds searched cities to chips bar, allows one-click switching | Saved up to 5 unique recent cities | **PASS** |
| **TC-09** | Production Build (`npm run build`) | Vite bundles assets with zero syntax/type errors | Generated optimized dist bundle in 15.14s | **PASS** |
| **TC-10** | Mobile Viewport (< 640px) | Adapts layout into a responsive single-column feed | Clean, scrollable mobile layout | **PASS** |
```

---

21. Instructions for Installing and Running the Project

Running via Terminal

1. Open PowerShell or Terminal.

2. Navigate to the project folder:

```
cd "weatherwise-week4-api-integration"
```

3. Install dependencies:

```
npm install
```

4. Start the Vite development server:

```
npm run dev
```

5. Open your browser at:

```
http://localhost:5173
```

6. Build for production:

```
npm run build
```

---

22. Screenshot Documentation

• **Main Dashboard:** `screenshots/main_dashboard.png` — Showcases the hero weather card, atmospheric metric cards, 24-hour hourly scroll, and 7-day extended forecast.

• **City Search & Weather:** `screenshots/search_success.png` — Displays successful geocoding and live data presentation for Tokyo.

• **Unit Conversion:** `screenshots/unit_toggle_fahrenheit.png` — Demonstrates real-time conversion to Fahrenheit across all cards.

• **City Disambiguation:** `screenshots/city_disambiguation.png` — Shows the search dropdown allowing selection between multiple matching cities.

• **Error Handling:** `screenshots/error_state.png` — Illustrates the resilient error banner when searching for non-existent locations.

• **Mobile Responsive View:** `screenshots/mobile_view.png` — Demonstrates full responsiveness on mobile screens.

---

23. Future Improvements

1. **Geolocation API Integration:** Use the browser's `navigator.geolocation` to automatically detect the user's current city on first load with their permission.

2. **Interactive Weather Radar Map:** Integrate Leaflet / OpenStreetMap with precipitation radar overlays.

3. **Severe Weather Alerts:** Incorporate Open-Meteo extreme weather warning notifications.

4. **Offline PWA Support:** Add Service Workers to cache recent forecasts for offline inspection.

---

24. Conclusion

The **WeatherWise** project demonstrates a complete implementation of modern React.js API integration and data handling. By pairing the Open-Meteo public APIs with clean asynchronous services, a dedicated custom hook (`useWeather`), responsive CSS design, and defensive error boundaries, the application delivers a robust, accessible, and high-performance weather dashboard suitable for production deployment and internship evaluation.